

Dokumentacja implementacyjna

Graficzny interfejs użytkownika – wizualizacja grafów (Java Swing)

Mateusz Kamieniecki
Jan Rękas

28 maja 2026

Spis treści

1	Przegląd architektury	3
2	Klasy modelu danych	3
2.1	graph.Vertex	3
2.2	graph.Edge	3
2.3	graph.Graph	4
3	Klasy widoku	4
3.1	graph.Bar (klasa abstrakcyjna)	4
3.2	graph.Footer (rozszerza Bar)	4
3.3	graph.GraphPanel (rozszerza JPanel)	5
3.3.1	Rysowanie	5
3.3.2	Interakcja myszą	5
3.3.3	Metody publiczne	5
3.4	graph.LeftPanel	6
4	Klasy kontrolera	6
4.1	graph.GraphSettingsCard (rozszerza JPanel)	6
4.1.1	Tryby pracy	6
4.1.2	Metoda onExecute()	7
4.1.3	Metoda runEngine()	7
4.1.4	Wyszukiwanie silnika – findEnginePath()	7
4.1.5	Metody pomocnicze (statyczne, dostępne dla innych klas)	8
4.2	graph.GraphInteractionPanel (rozszerza JPanel)	8
4.2.1	Metoda activate(String inputFilePath)	8
4.2.2	Metoda savePositions()	8
5	Klasa graph.GraphFileReader	8
5.1	readTextEdges(String path)	9
5.2	readTextVertices(String path)	9
5.3	writeTextOutput(Collection<Vertex> vertices, String path)	9
5.4	readBinaryInput(String path)	9

6	Klasy pomocnicze i kontener główny	10
6.1	graph.MainFrame (rozszerza JFrame)	10
6.2	graph.Main	10
7	Wątkowość	10
8	Spójność komponentów i obsługa błędów	11
9	Integracja z silnikiem C	11

1 Przegląd architektury

Projekt Java realizuje graficzny interfejs użytkownika do wizualizacji grafów, współpracując z zewnętrznym silnikiem obliczeniowym napisanym w języku C. Całość opiera się na bibliotece **Java Swing** i jest zorganizowana w pakiecie **graph**.

Architektura oprogramowania jest zbliżona do wzorca **MVC** (Model–Widok–Kontroler):

- **Model** – klasy **Graph**, **Vertex**, **Edge** przechowują strukturę grafu bez żadnej logiki prezentacji.
- **Widok** – **GraphPanel** (rysowanie grafu), **Footer** (pasek statusu), klasy panelów bocznych.
- **Kontroler** – **GraphSettingsCard** i **GraphInteractionPanel** reagują na akcje użytkownika, modyfikują model i zlecają odświeżenie widoku.

Komunikacja z silnikiem C odbywa się przez **ProcessBuilder** – zewnętrzny plik **engine.exe** przyjmuje graf wejściowy i zwraca obliczone pozycje wierzchołków w formacie tekstowym lub binarnym.

2 Klasy modelu danych

2.1 graph.Vertex

Klasa modelowa reprezentująca pojedynczy wierzchołek grafu.

```
1 public class Vertex {  
2     public int id;  
3     public double x, y;  
4 }
```

id Unikalny identyfikator całkowitoliczbowy wierzchołka.

x, y Współrzędne ekranowe (w pikselach logicznych) wykorzystywane przez **GraphPanel** do rysowania i przeciągania.

Współrzędne są modyfikowane bezpośrednio przez **GraphPanel** (podczas przeciągania myszą) oraz przez **GraphSettingsCard** po wczytaniu pozycji z pliku. Klasa *nie* przechowuje listy sąsiedztwa – relacje między wierzchołkami są modelowane wyłącznie przez krawędzie.

2.2 graph.Edge

Klasa modelowa reprezentująca pojedynczą krawędź grafu.

```
1 public class Edge {  
2     public String name;  
3     public Vertex from, to;  
4     public double weight;  
5 }
```

<code>name</code>	Etykieta krawędzi wyświetlana na środku odcinka.
<code>from, to</code>	Referencje do wierzchołków końcowych (krawędź jest nieskierowana).
<code>weight</code>	Waga krawędzi wyświetlana obok nazwy.

Krawędź nie przechowuje własnych współrzędnych – przy rysowaniu odczytuje je bezpośrednio z pól `x`, `y` obiektów `Vertex`.

2.3 graph.Graph

Główna klasa modelu danych. Przechowuje listy wierzchołków i krawędzi, udostępnia metody do ich dodawania, usuwania i wyszukiwania.

Kluczowe metody:

- `addVertex(Vertex v)` – dodaje wierzchołek do grafu.
- `addEdge(Edge e)` – dodaje krawędź do grafu.
- `getVertexById(int id)` – zwraca wierzchołek o podanym identyfikatorze lub `null`.
- `getVertices()` / `getEdges()` – zwracają listy (`ArrayList`) wierzchołków i krawędzi.
- `clear()` – usuwa wszystkie wierzchołki i krawędzie.

Klasa jest czystym modelem danych – nie zawiera logiki rysowania ani algorytmów. Listy `ArrayList` są współdzielone referencją z `GraphPanel`, więc każda modyfikacja modelu jest natychmiast widoczna po wywołaniu `repaint()`.

3 Klasy widoku

3.1 graph.Bar (klasa abstrakcyjna)

Klasa bazowa dla wszystkich komponentów paskowych interfejsu (dolny pasek statusu, potencjalny pasek narzędziowy). Dostarcza wspólnych stałych stylistycznych:

```

1 protected static final Color BG           = new Color(18, 18, 28);
2 protected static final Color BORDER_COL  = new Color(40, 40, 60);
3 protected static final Color TEXT        = new Color(200, 200, 215);
4 protected static final Font  FONT        = new Font("Segoe UI", Font
    .PLAIN, 12);

```

Konstruktor ustawia tło i czcionkę. Klasa nie zawiera logiki biznesowej – służy wyłącznie ujednoliceniu wyglądu pasków.

3.2 graph.Footer (rozszerza Bar)

Dolny pasek statusu informujący użytkownika o bieżących akcjach (ładowanie grafu, postęp algorytmu, błędy wejścia/wyjścia).

Kluczowa metoda:

```

1 public void setStatus(String message) {
2     SwingUtilities.invokeLater(() -> label.setText(message));
3 }

```

Użycie `SwingUtilities.invokeLater()` gwarantuje bezpieczeństwo wątkowe – metoda może być wywoływana z dowolnego wątku, w tym z wątku `SwingWorker` obsługującego długotrwałe obliczenia silnika.

Komponent zawiera `JLabel` z domyślnym tekstem „*Gotowy*”. Górna krawędź paska jest oddzielona linią o kolorze `BORDER_COL`.

3.3 graph.GraphPanel (rozszerza JPanel)

Centralny komponent graficzny odpowiedzialny za wizualizację grafu i interakcję myszą. Rysowanie jest zaimplementowane samodzielnie przez nadpisanie metody `paintComponent()`.

3.3.1 Rysowanie

```

1 @Override
2 protected void paintComponent(Graphics g) {
3     super.paintComponent(g);
4     Graphics2D g2 = (Graphics2D) g;
5     g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
6                         RenderingHints.VALUE_ANTIALIAS_ON);
7     g2.translate(panX, panY);
8     g2.scale(scale, scale);
9     // rysowanie krawędzi, nast pnie wierzcho k w
10 }

```

Transformacja `translate` + `scale` pozwala na płynne przesuwanie i skalowanie widoku bez modyfikacji współrzędnych modelu. Wierzchołki rysowane są jako wypełnione okręgi (promień `RADIUS = 12 px`), krawędzie jako odcinki z opcjonalnym tekstem pośrodku.

3.3.2 Interakcja myszą

Panel obsługuje trzy tryby interakcji:

- **Przeciąganie wierzchołków** (lewy przycisk myszy) – po kliknięciu w obszarze wierzchołka (`distance <= RADIUS`) aktualizuje `v.x`, `v.y` w modelu i wywołuje `repaint()`.
- **Przesuwanie widoku (pan)** (środkowy przycisk myszy) – aktualizuje `panX`, `panY`.
- **Zoom** (kółko myszy) – mnoży lub dzieli `scale` przez `1.1`.

3.3.3 Metody publiczne

`setZoom(double)` Ustawia poziom powiększenia i wywołuje `repaint()`.

`resetView()` Zeruje przesunięcie widoku (`panX = panY = 0`).

<code>fitToView()</code>	Automatycznie dobiera zoom i przesunięcie tak, by cały graf zmieścił się w oknie z 80 px marginesem. Oblicza <code>minX</code> , <code>maxX</code> , <code>minY</code> , <code>maxY</code> i wyznacza <code>scale = min(scaleX, scaleY)</code> , zachowując proporcje.
<code>setShowVertexDecorators(boolean)</code>	Pokazuje lub ukrywa identyfikatory wierzchołków.
<code>setShowEdgeDecorators(boolean)</code>	Pokazuje lub ukrywa etykiety i wagi krawędzi.
<code>moveVertex(int id, double dx, double dy)</code>	Przesuwa wierzchołek o podanym ID o wektor $(\Delta x, \Delta y)$.

3.4 graph.LeftPanel

Panel boczny działający jako kontener z układem kartowym (`CardLayout`). Zarządza przełączaniem między dwoma kartami:

- „**settings**” – karta `GraphSettingsCard` (wybór pliku, trybu, algorytmu i uruchomienie obliczania).
- „**interaction**” – karta `GraphInteractionPanel` (opcje wyświetlania, sterowanie widokiem, zapis pozycji).

Metody `showCard(String)` i `getInteractionPanel()` umożliwiają zewnętrznym komponentom przełączanie widoku.

4 Klasy kontrolera

4.1 graph.GraphSettingsCard (rozszerza JPanel)

Karta ustawień – pierwszy ekran widoczny po uruchomieniu. Umożliwia:

1. Wskazanie pliku wejściowego z listą krawędzi grafu.
2. Wybór trybu działania (algorytm lub wczytanie gotowych pozycji).
3. Uruchomienie silnika obliczeniowego lub odczyt pozycji z pliku.

4.1.1 Tryby pracy

<code>radioFrucht</code>	Algorytm Fruchtermana-Reingolda (domyślny).
<code>radioTriang</code>	Algorytm triangulacji.
<code>radioLoadTxt</code>	Wczytanie pozycji z pliku <code>.txt</code> .
<code>radioLoadBin</code>	Wczytanie pozycji z pliku binarnego.

Wybór trybu dynamicznie pokazuje lub ukrywa sekcję `loadSection` (pole ścieżki do pliku pozycji) i `algoSection` (wybór formatu zapisu) za pomocą `Runnable sync`.

4.1.2 Metoda onExecute()

Główny punkt wejścia po kliknięciu przycisku „Wykonaj”:

1. Waliduje ścieżkę wejściową (niepusta, plik istnieje).
2. Wczytuje krawędzie przez `GraphFileReader.readTextEdges()`.
3. W zależności od trybu:
 - tryby `LoadTxt` / `LoadBin` – wywołuje `applyPositionsTxt()` lub `applyPositionsBin()`, następnie przełącza kartę;
 - tryby algorytmiczne – wywołuje `runEngine()`.

4.1.3 Metoda runEngine()

Uruchamia silnik C w tle przy użyciu `SwingWorker`:

```
1 SwingWorker<Void, String> worker = new SwingWorker<>() {
2     @Override
3     protected Void doInBackground() throws Exception {
4         Process proc = new ProcessBuilder(cmd)
5             .redirectErrorStream(true).start();
6         // odczyt stdout i aktualizacja footer przez publish()
7         int code = proc.waitFor();
8         if (code != 0) throw new IOException(...);
9         return null;
10    }
11    @Override
12    protected void done() {
13        // wczytanie wynik w, prze czenie karty
14    }
15 };
16 worker.execute();
```

Zastosowanie `SwingWorker` zapewnia, że interfejs nie blokuje się podczas obliczeń. Metoda `process()` publikuje linie `stdout` silnika do `Footer`, a `done()` wczytuje wyniki i przełącza na kartę interakcji.

4.1.4 Wyszukiwanie silnika – findEnginePath()

Metoda statyczna przeszukuje kilka kandydujących ścieżek:

```
1 private static String findEnginePath() {
2     String[] candidates = {
3         "bin/engine/engine.exe",
4         "app/bin/engine/engine.exe",
5         "../bin/engine/engine.exe",
6     };
7     for (String c : candidates)
8         if (new File(c).exists()) return new File(c).
9             getAbsolutePath();
10    return new File("bin/engine/engine.exe").getAbsolutePath();
11 }
```

4.1.5 Metody pomocnicze (statyczne, dostępne dla innych klas)

<code>styledButton(String)</code>	Tworzy przycisk z ciemnym motywem.
<code>styledField()</code>	Tworzy pole tekstowe zgodne z motywem.
<code>styledRadio(String)</code>	Tworzy radio button z motywem.
<code>styledCheck(String)</code>	Tworzy checkbox z motywem.
<code>sectionLabel(String)</code>	Tworzy szary nagłówek sekcji.
<code>box()</code>	Tworzy <code>JPanel</code> z <code>BoxLayout.Y_AXIS</code> .
<code>vgap(int)</code>	Tworzy pionowy odstęp (<code>Box.createVerticalStrut</code>).
<code>separator()</code>	Tworzy wizualną linię separatora.
<code>describe(Throwable)</code>	Zwraca czytelny komunikat wyjątku.

Metody te są `static`, dzięki czemu `GraphInteractionPanel` importuje je statycznie i zachowuje spójny wygląd bez duplikacji kodu.

4.2 graph.GraphInteractionPanel (rozszerza JPanel)

Karta interakcji – aktywna po załadowaniu grafu. Zawiera trzy sekcje:

Wyświetlanie	Dwa checkboxy: identyfikatory wierzchołków i wagi krawędzi; każdy wywołuje odpowiednią metodę <code>setShow*()</code> na <code>GraphPanel</code> .
Widok	Trzy przyciski: <i>Resetuj zoom</i> (<code>graphPanel.setZoom(1.0)</code>), <i>Wyśrodkuj widok</i> (<code>graphPanel.resetView()</code>) oraz <i>Dopasuj do widoku</i> (<code>graphPanel.fitToView()</code>).
Eksport	Przycisk <i>Zapisz zmianę pozycji</i> , wywołujący metodę <code>savePositions()</code> .

4.2.1 Metoda activate(String inputFilePath)

Wywoływana przez `GraphSettingsCard` tuż przed przełączeniem karty. Zapamiętuje ścieżkę pliku wejściowego, potrzebną do generowania nazwy pliku wyjściowego.

4.2.2 Metoda savePositions()

Generuje nazwę pliku wyjściowego (`<baseName>_newPositions.txt`) w tym samym katalogu co plik wejściowy i zapisuje aktualne współrzędne wierzchołków przez `GraphFileReader.writeText()`.

5 Klasa graph.GraphFileReader

Klasa narzędziowa (wszystkie metody `static`) obsługująca wejście i wyjście plików grafu. Oddziela logikę parsowania od kontrolera.

5.1 readTextEdges(String path)

Format wejściowy: każda linia zawiera <nazwa> <fromId> <toId> <waga> oddzielone białymi znakami.

```
1 public static List<Edge> readTextEdges(String path) throws  
    IOException
```

Działanie:

1. Wczytuje plik linia po linii przez `BufferedReader`.
2. Pomija puste linie.
3. Waliduje, że każda linia ma co najmniej 4 tokeny; przy błędzie rzuca `IOException` z numerem linii.
4. Deduplikuje wierzchołki przez `LinkedHashMap<Integer, Vertex>` – ten sam wierzchołek nie jest tworzony wielokrotnie, a referencja jest współdzielona między krawędziami.
5. Zwraca niepustą listę krawędzi; jeśli jest pusta – rzuca `IOException`.

5.2 readTextVertices(String path)

Format wejściowy: każda linia zawiera <id> <x> <y>.

```
1 public static List<Vertex> readTextVertices(String path) throws  
    IOException
```

Używany do wczytywania pozycji wierzchołków z pliku `.txt` zapisanego przez silnik C. Akceptuje zarówno kropkę, jak i przecinek jako separator dziesiętny (`replace(',',',', '.',')'`).

5.3 writeTextOutput(Collection<Vertex> vertices, String path)

```
1 public static void writeTextOutput(Collection<Vertex> vertices,  
    String path)  
2     throws IOException
```

Zapisuje pozycje wierzchołków do pliku tekstowego w formacie `%d %.2f %.2f` (id, x, y) przez `PrintWriter`.

5.4 readBinaryInput(String path)

```
1 public static List<Vertex> readBinaryInput(String path) throws  
    IOException
```

Wczytuje plik binarny wygenerowany przez silnik C. Format rekordu (20 bajtów, little-endian):

Offset	Typ	Znaczenie
0	int (4 B)	identyfikator wierzchołka
4	double (8 B)	współrzędna x
12	double (8 B)	współrzędna y

Odczyt realizowany przez `ByteBuffer.wrap(allBytes).order(ByteOrder.LITTLE_ENDIAN)`.
Pętla czyta kolejne rekordy dopóki bufor ma co najmniej 20 bajtów.

6 Klasy pomocnicze i kontener główny

6.1 graph.MainFrame (rozszerza JFrame)

Kontener główny okna aplikacji.

Odpowiedzialność:

- Tworzenie instancji modelu (`Graph`) i przykładowych danych (wierzchołki i krawędzie do testów).
- Inicjalizacja widoków: `GraphPanel`, `Footer`, `LeftPanel`.
- Ustawienie `BorderLayout`: `Footer` na dole, `LeftPanel` po lewej, `GraphPanel` na środku.
- Konfiguracja okna: tytuł, rozmiar, `EXIT_ON_CLOSE`, wyśrodkowanie na ekranie.

Uwaga: Zgodnie z wymaganiami należy dodać pasek menu (`JMenuBar`) z opcjami: wczytanie grafu z pliku tekstowego i binarnego, zapis wyników oraz pomoc.

6.2 graph.Main

Klasa startowa. Metoda `main` uruchamia GUI w wątku EDT:

```

1 public static void main(String[] args) {
2     SwingUtilities.invokeLater(() -> new MainFrame().setVisible(
3         true));
4 }
```

Nie zawiera logiki biznesowej.

7 Wątkowość

Projekt przestrzega zasad bezpieczeństwa wątkowego Swing:

- Wszystkie operacje na komponentach GUI odbywają się w wątku EDT.
- `Footer.setStatus()` używa `SwingUtilities.invokeLater()` i może być bezpiecznie wywoływana z dowolnego wątku.
- Długotrwałe operacje (uruchomienie silnika C, oczekiwanie na wyniki) wykonywane są w `SwingWorker.doInBackground()`, natomiast aktualizacja interfejsu – w `process()` i `done()`, które działają na EDT.
- Bezpośrednie modyfikacje modelu (`Vertex.x`, `Vertex.y`) podczas przeciągania myszą odbywają się w EDT (obsługa zdarzeń myszy).

8 Spójność komponentów i obsługa błędów

- `GraphSettingsCard.onExecute()` waliduje ścieżki przed każdą operacją i wyświetla `JOptionPane.ERROR_MESSAGE` z opisem błędu.
- Metoda `describe(Throwable)` (statyczna w `GraphSettingsCard`) wyodrębnia czytelny komunikat z wyjątku, używana w całym pakiecie.
- `GraphFileReader` rzuca `IOException` z numerem linii przy błędzie parsowania, co pozwala użytkownikowi szybko znaleźć problem w pliku wejściowym.
- Po pomyślnym wczytaniu grafu panel automatycznie przełącza się na kartę interakcji (`leftPanel.showCard("interaction")`), zapobiegając operowaniu na pustym widoku.
- Przycisk *Wróć do ustawień* w `GraphInteractionPanel` zawsze umożliwia powrót bez utraty wczytanego grafu.

9 Integracja z silnikiem C

Silnik obliczeniowy (`engine.exe`) jest osobnym procesem uruchamianym przez `ProcessBuilder`. Interfejs komunikacji:

```
1 engine.exe <sciezka_grafu> -a <algorytm> -o <sciezka_wyjsciowa> [-t  
  | -b]
```

- | | |
|----|---|
| -a | Algorytm: <code>fruchterman</code> lub <code>triangulation</code> . |
| -o | Ścieżka bazowa pliku wyjściowego (bez rozszerzenia przy formacie binarnym). |
| -t | Zapis w formacie tekstowym (plik <code>.txt</code>). |
| -b | Zapis w formacie binarnym. |

Silnik zapisuje wyniki w podkatalogu `positions/` obok pliku grafu wejściowego. Po zakończeniu procesu z kodem 0 `GraphSettingsCard` wczytuje wyniki i stosuje je do modelu przez `applyPositionsTxt()` lub `applyPositionsBin()`.